

CLOUD COMPUTING LAB

Experiment 2 — Client–Server Communication Between Two Virtual Machines

Implementation of a Basic TCP-Based Chat Application Between Two Virtual Machines

Environment: Oracle VirtualBox + Ubuntu

Architecture: One VM as TCP Server and one VM as TCP Client

Network: CloudLab-Network (NAT Network)

Submitted as a practical implementation record

Experiment Overview

This record documents the implementation sequence: creation of two Linux virtual machines, configuration of a common virtual network, verification of IP addressing and connectivity, creation of TCP server and client programs, establishment of the TCP connection, and successful message exchange between the two VMs.

Evidence Sequence

1. Virtual machines → 2. NAT network → 3. Network configuration → 4. IP addresses → 5. Ping connectivity → 6. TCP server → 7. TCP client → 8. Server waiting → 9. Connection established → 10. Chat communication.

1. Create and prepare two Linux virtual machines

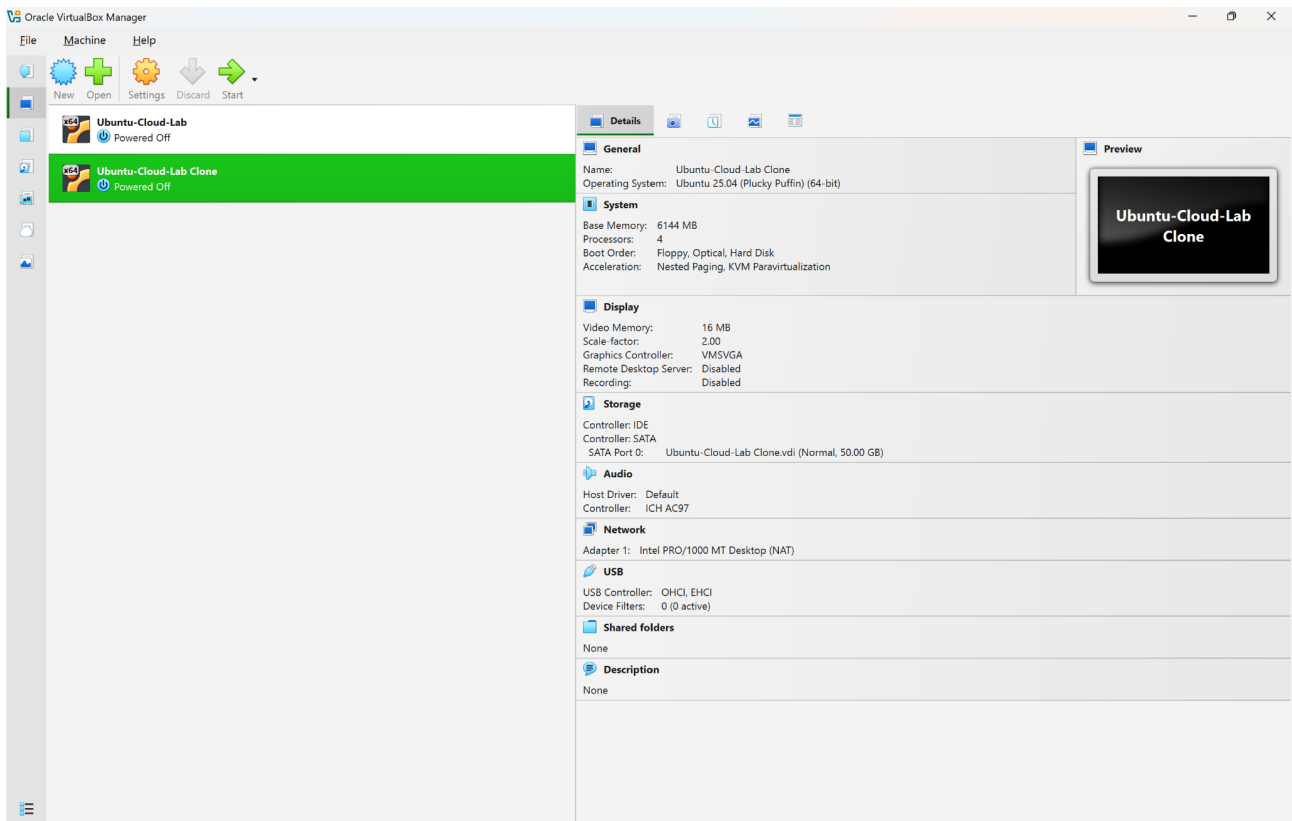


Figure 1: Creation of two Linux virtual machines using VirtualBox.

2. Create the NAT network

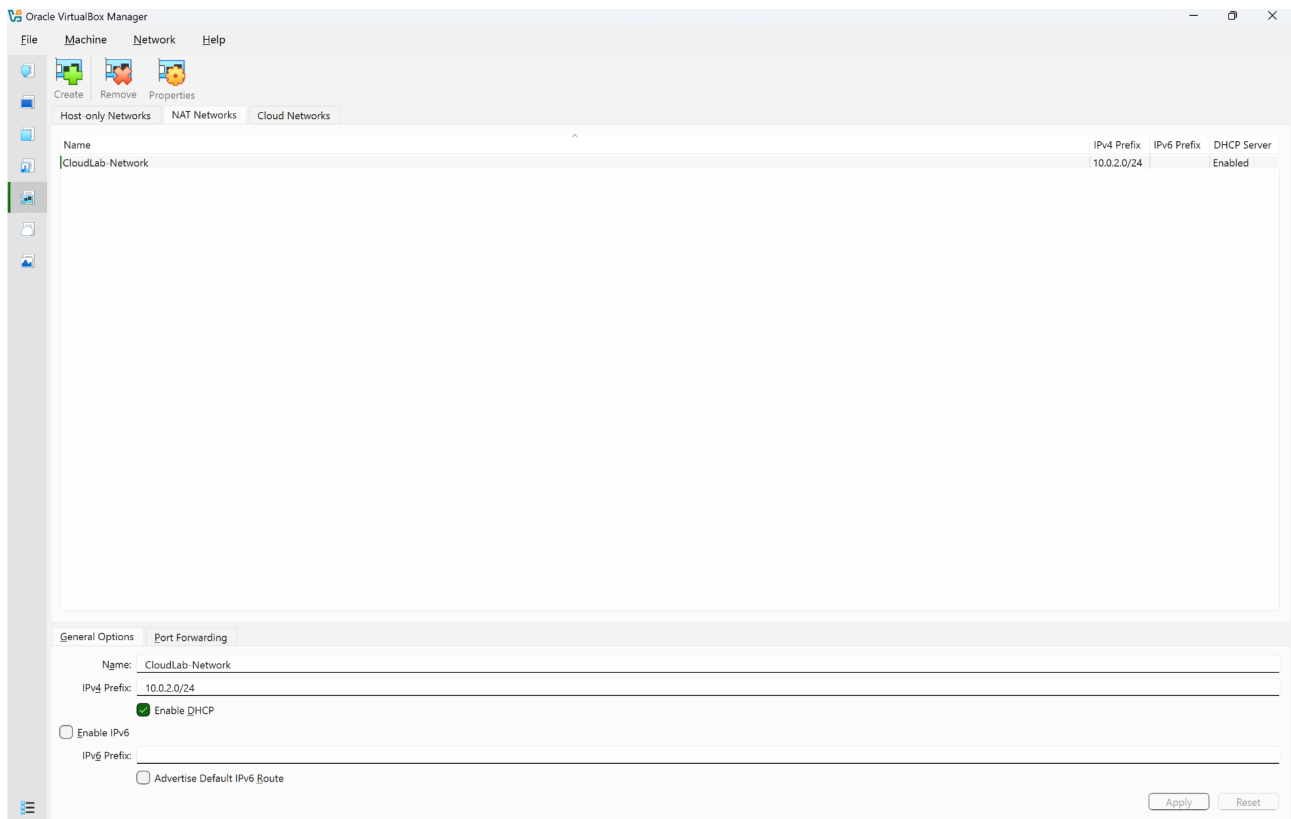


Figure 2: Creation of NAT network for communication between the virtual machines.

3. Configure both VMs on the same virtual network

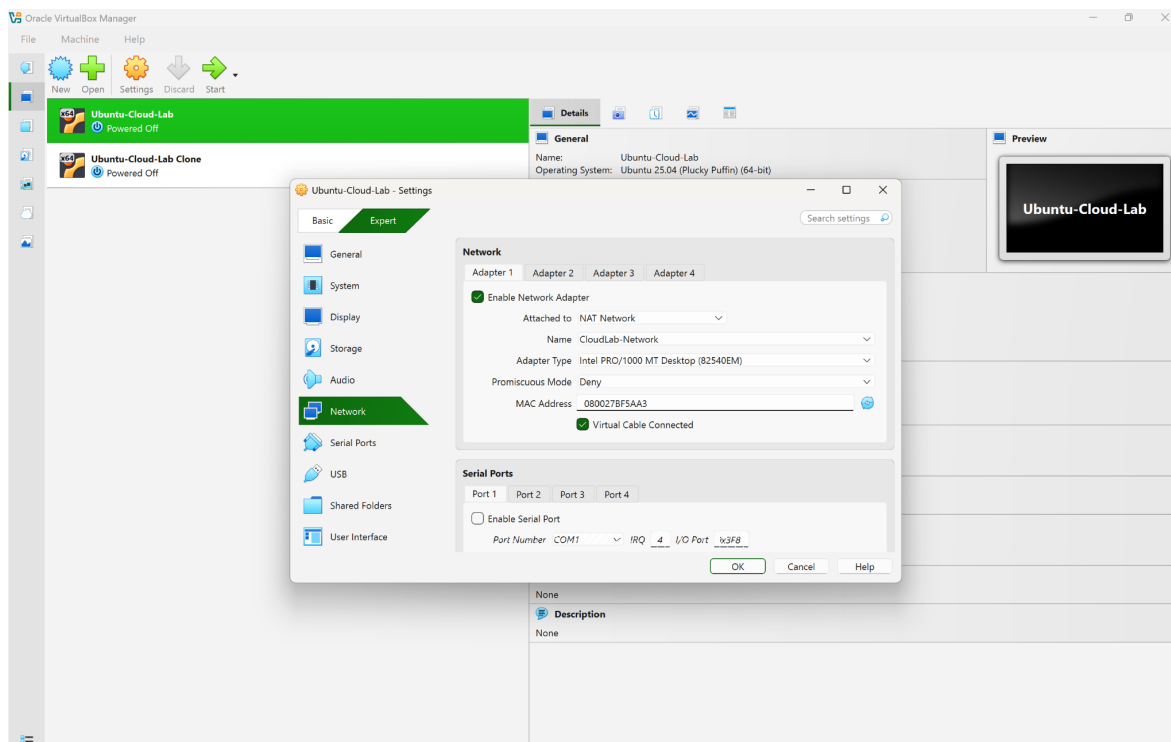


Figure 3: Configuration of both Linux VMs on the same virtual network — Server VM.

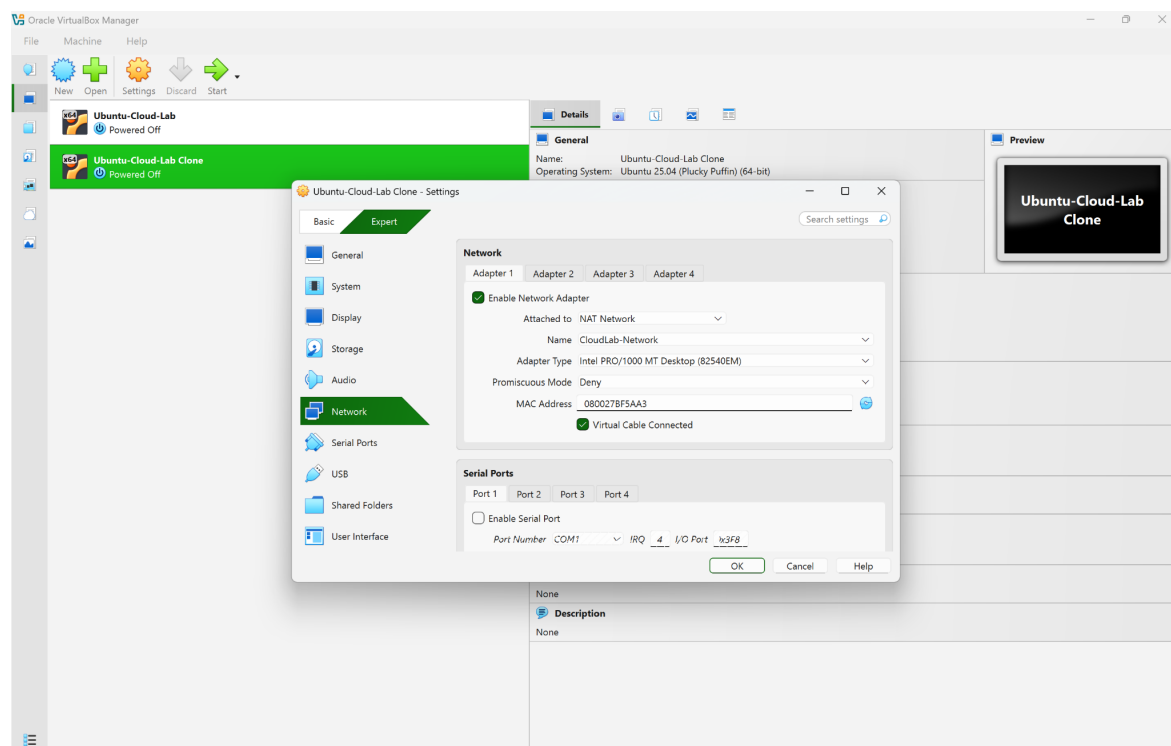
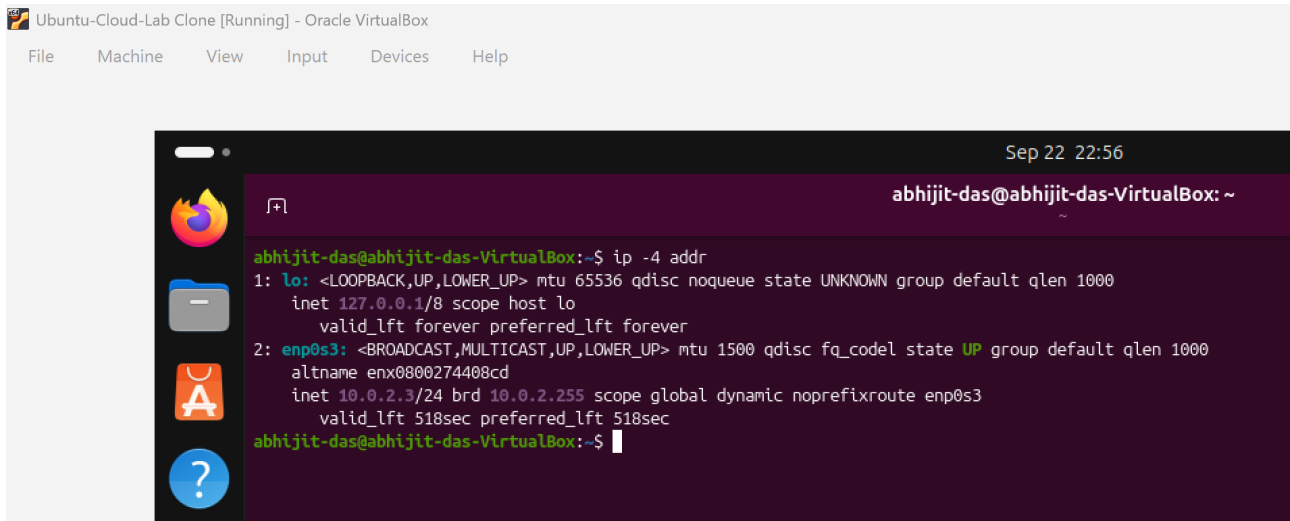


Figure 3: Configuration of both Linux VMs on the same virtual network — Client VM.

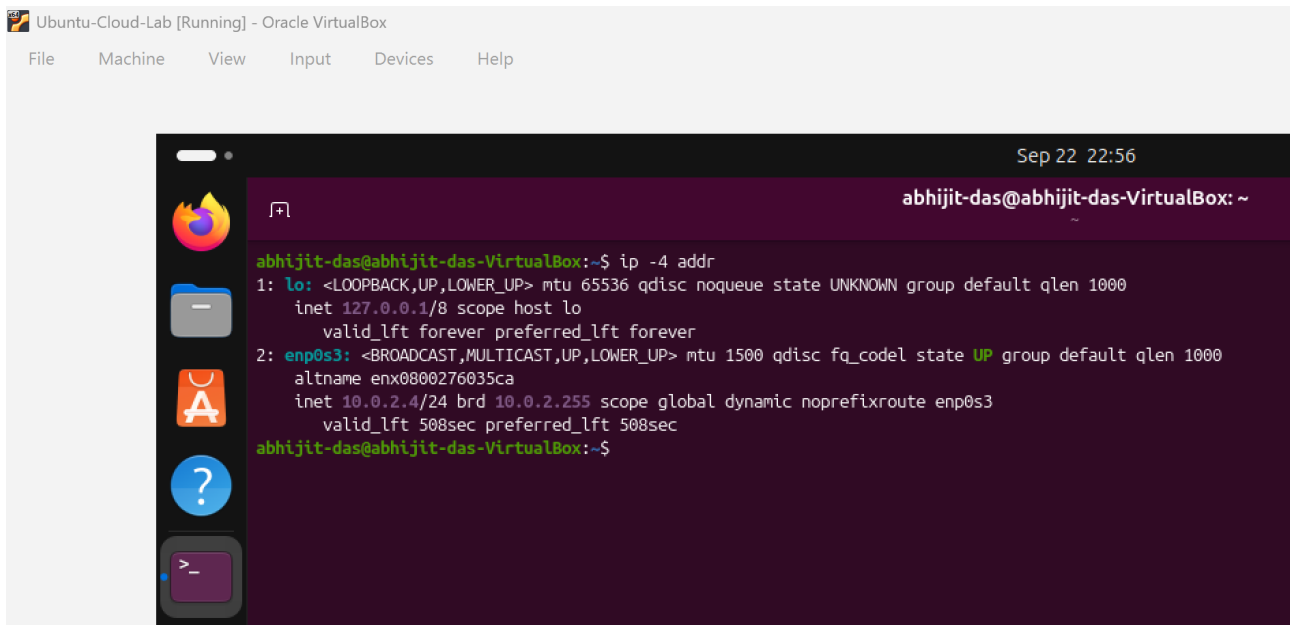
4. Check the IP addresses of both VMs



The screenshot shows a terminal window titled "Ubuntu-Cloud-Lab Clone [Running] - Oracle VirtualBox". The terminal output shows the command `ip -4 addr` and its output:

```
abhihit-das@abhihit-das-VirtualBox:~$ ip -4 addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
   inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
   altname enx0800274408cd
   inet 10.0.2.3/24 brd 10.0.2.255 scope global dynamic noprefixroute enp0s3
       valid_lft 518sec preferred_lft 518sec
abhihit-das@abhihit-das-VirtualBox:~$
```

Figure 4: IP address displayed on the Client VM.

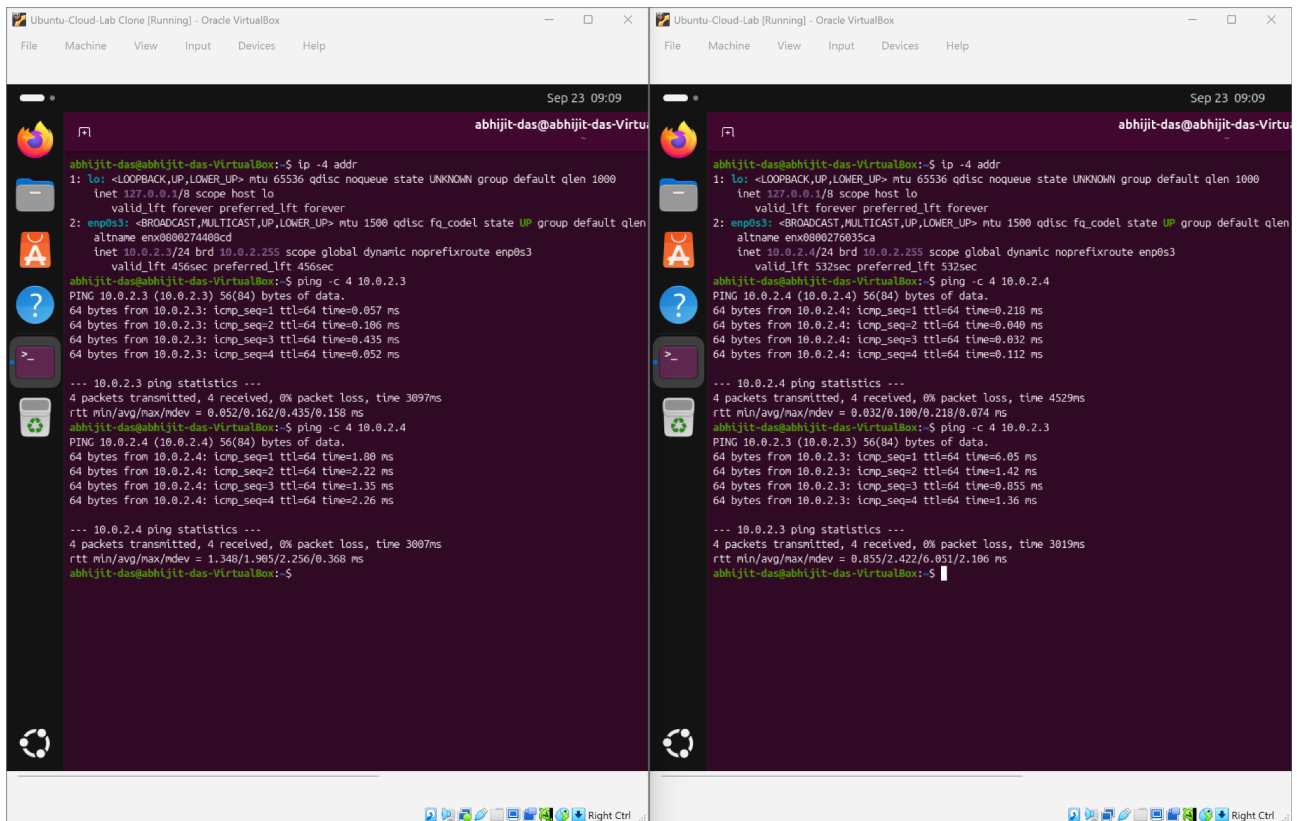


The screenshot shows a terminal window titled "Ubuntu-Cloud-Lab [Running] - Oracle VirtualBox". The terminal output shows the command `ip -4 addr` and its output:

```
abhihit-das@abhihit-das-VirtualBox:~$ ip -4 addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
   inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
   altname enx0800276035ca
   inet 10.0.2.4/24 brd 10.0.2.255 scope global dynamic noprefixroute enp0s3
       valid_lft 508sec preferred_lft 508sec
abhihit-das@abhihit-das-VirtualBox:~$
```

Figure 4: IP address displayed on the Server VM.

5. Test connectivity using ping



The image shows two side-by-side terminal windows from an Oracle VM VirtualBox environment. Both windows are titled 'Ubuntu-Cloud-Lab Clone [Running] - Oracle VirtualBox' and show a terminal session for a user named 'abhijit-das' on a machine named 'abhijit-das-VirtualBox'. The date and time in the top right corner of both windows is 'Sep 23 09:09'.

Left Terminal Window:

```
abhijit-das@abhijit-das-VirtualBox:~$ ip -4 addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen
    altnam enx8080274408cd
    inet 10.0.2.3/24 brd 10.0.2.255 scope global dynamic noprefixroute enp0s3
        valid_lft 456sec preferred_lft 456sec
abhijit-das@abhijit-das-VirtualBox:~$ ping -c 4 10.0.2.3
PING 10.0.2.3 (10.0.2.3) 56(84) bytes of data.
64 bytes from 10.0.2.3: icmp_seq=1 ttl=64 time=0.057 ms
64 bytes from 10.0.2.3: icmp_seq=2 ttl=64 time=0.106 ms
64 bytes from 10.0.2.3: icmp_seq=3 ttl=64 time=0.435 ms
64 bytes from 10.0.2.3: icmp_seq=4 ttl=64 time=0.052 ms

--- 10.0.2.3 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3097ms
rtt min/avg/max/mdev = 0.052/0.162/0.435/0.158 ms
abhijit-das@abhijit-das-VirtualBox:~$ ping -c 4 10.0.2.4
PING 10.0.2.4 (10.0.2.4) 56(84) bytes of data.
64 bytes from 10.0.2.4: icmp_seq=1 ttl=64 time=1.80 ms
64 bytes from 10.0.2.4: icmp_seq=2 ttl=64 time=2.22 ms
64 bytes from 10.0.2.4: icmp_seq=3 ttl=64 time=1.35 ms
64 bytes from 10.0.2.4: icmp_seq=4 ttl=64 time=2.26 ms

--- 10.0.2.4 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3007ms
rtt min/avg/max/mdev = 1.348/1.905/2.256/0.368 ms
abhijit-das@abhijit-das-VirtualBox:~$
```

Right Terminal Window:

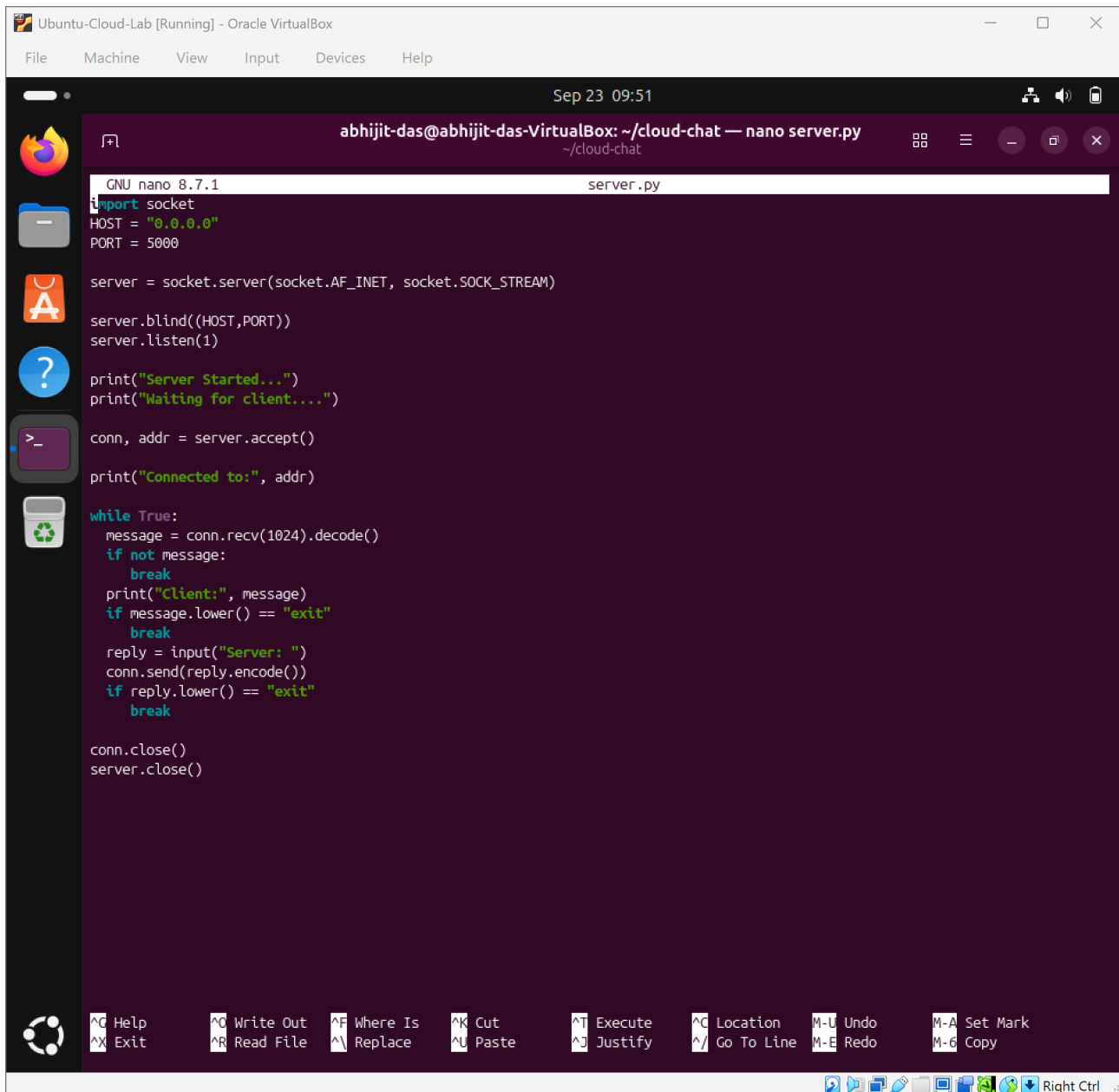
```
abhijit-das@abhijit-das-VirtualBox:~$ ip -4 addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen
    altnam enx8080276035ca
    inet 10.0.2.4/24 brd 10.0.2.255 scope global dynamic noprefixroute enp0s3
        valid_lft 532sec preferred_lft 532sec
abhijit-das@abhijit-das-VirtualBox:~$ ping -c 4 10.0.2.4
PING 10.0.2.4 (10.0.2.4) 56(84) bytes of data.
64 bytes from 10.0.2.4: icmp_seq=1 ttl=64 time=0.218 ms
64 bytes from 10.0.2.4: icmp_seq=2 ttl=64 time=0.040 ms
64 bytes from 10.0.2.4: icmp_seq=3 ttl=64 time=0.032 ms
64 bytes from 10.0.2.4: icmp_seq=4 ttl=64 time=0.112 ms

--- 10.0.2.4 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 4529ms
rtt min/avg/max/mdev = 0.032/0.100/0.218/0.074 ms
abhijit-das@abhijit-das-VirtualBox:~$ ping -c 4 10.0.2.3
PING 10.0.2.3 (10.0.2.3) 56(84) bytes of data.
64 bytes from 10.0.2.3: icmp_seq=1 ttl=64 time=6.05 ms
64 bytes from 10.0.2.3: icmp_seq=2 ttl=64 time=1.42 ms
64 bytes from 10.0.2.3: icmp_seq=3 ttl=64 time=0.855 ms
64 bytes from 10.0.2.3: icmp_seq=4 ttl=64 time=1.36 ms

--- 10.0.2.3 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3019ms
rtt min/avg/max/mdev = 0.855/2.422/6.051/2.106 ms
abhijit-das@abhijit-das-VirtualBox:~$
```

Figure 5: Testing connectivity between the two virtual machines using ping.

6. Create the TCP server program



The screenshot shows a terminal window titled 'Ubuntu-Cloud-Lab [Running] - Oracle VirtualBox'. The terminal is running the nano text editor, editing a file named 'server.py' in the directory '~/.cloud-chat'. The code in the file is a Python TCP server program. The terminal window has a menu bar with 'File', 'Machine', 'View', 'Input', 'Devices', and 'Help'. The terminal output shows the program's execution, including the server starting, waiting for a client, and handling a connection. The bottom of the terminal window displays a list of keyboard shortcuts for nano, such as ^C for Help, ^X for Exit, ^O for Write Out, and ^R for Read File.

```
GNU nano 8.7.1 server.py
import socket
HOST = "0.0.0.0"
PORT = 5000

server = socket.server(socket.AF_INET, socket.SOCK_STREAM)

server.bind((HOST,PORT))
server.listen(1)

print("Server Started...")
print("Waiting for client....")

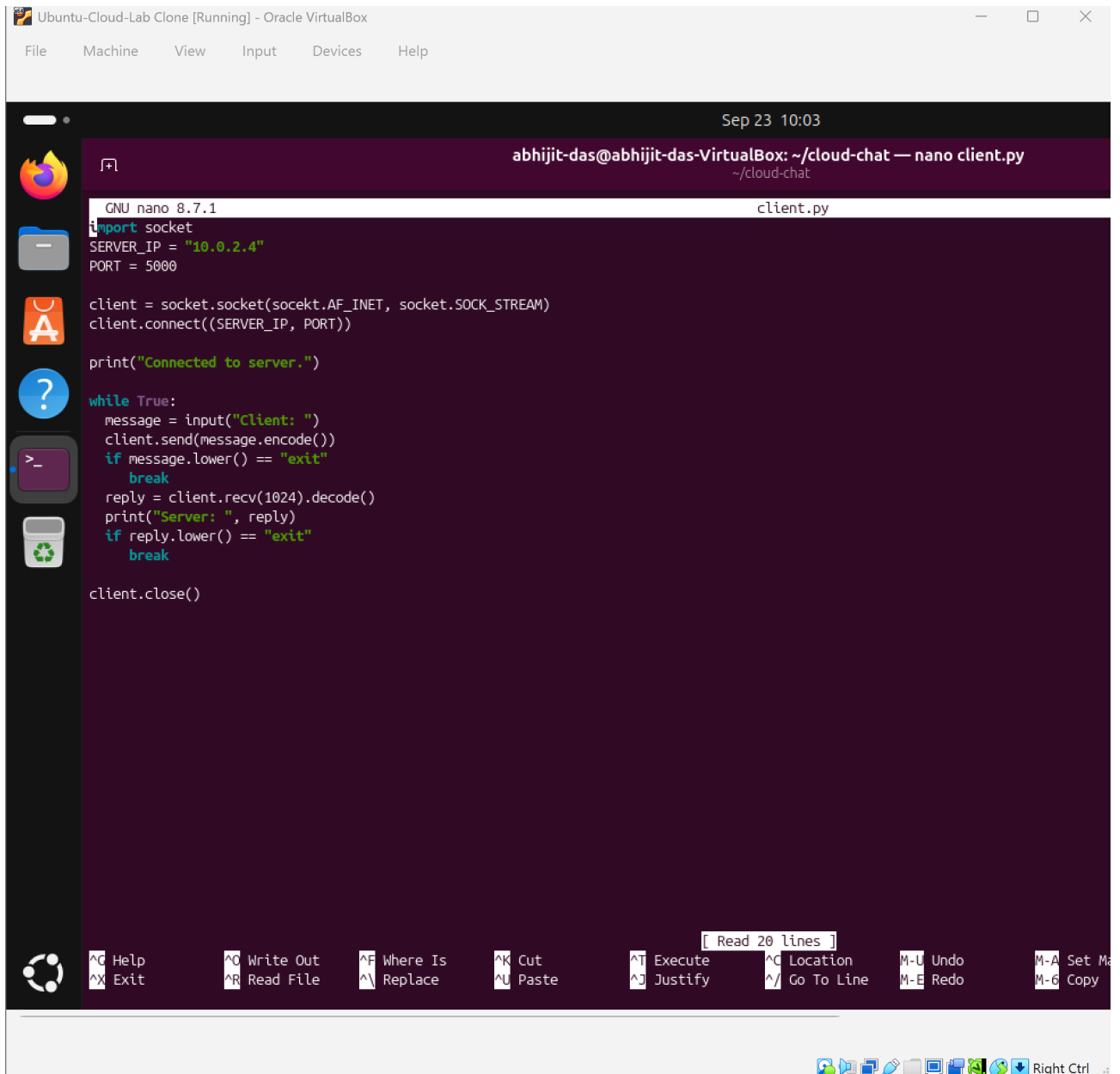
conn, addr = server.accept()
print("Connected to:", addr)

while True:
    message = conn.recv(1024).decode()
    if not message:
        break
    print("Client:", message)
    if message.lower() == "exit":
        break
    reply = input("Server: ")
    conn.send(reply.encode())
    if reply.lower() == "exit":
        break

conn.close()
server.close()
```

Figure 6: TCP server program implemented on the Server VM.

7. Create the TCP client program



The screenshot shows a VirtualBox window titled "Ubuntu-Cloud-Lab Clone [Running] - Oracle VirtualBox". Inside the VM, a terminal window is open, displaying the nano 8.7.1 editor. The editor is editing a file named "client.py" located at "~/cloud-chat". The code in the file is as follows:

```
import socket
SERVER_IP = "10.0.2.4"
PORT = 5000

client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect((SERVER_IP, PORT))

print("Connected to server.")

while True:
    message = input("Client: ")
    client.send(message.encode())
    if message.lower() == "exit":
        break
    reply = client.recv(1024).decode()
    print("Server: ", reply)
    if reply.lower() == "exit":
        break

client.close()
```

The terminal window also shows the prompt "abhihit-das@abhihit-das-VirtualBox: ~/cloud-chat — nano client.py" and the time "Sep 23 10:03". The bottom of the window displays a status bar with various keyboard shortcuts and a "Read 20 lines" indicator.

Figure 7: TCP client program implemented on the Client VM.

8. Start the TCP server

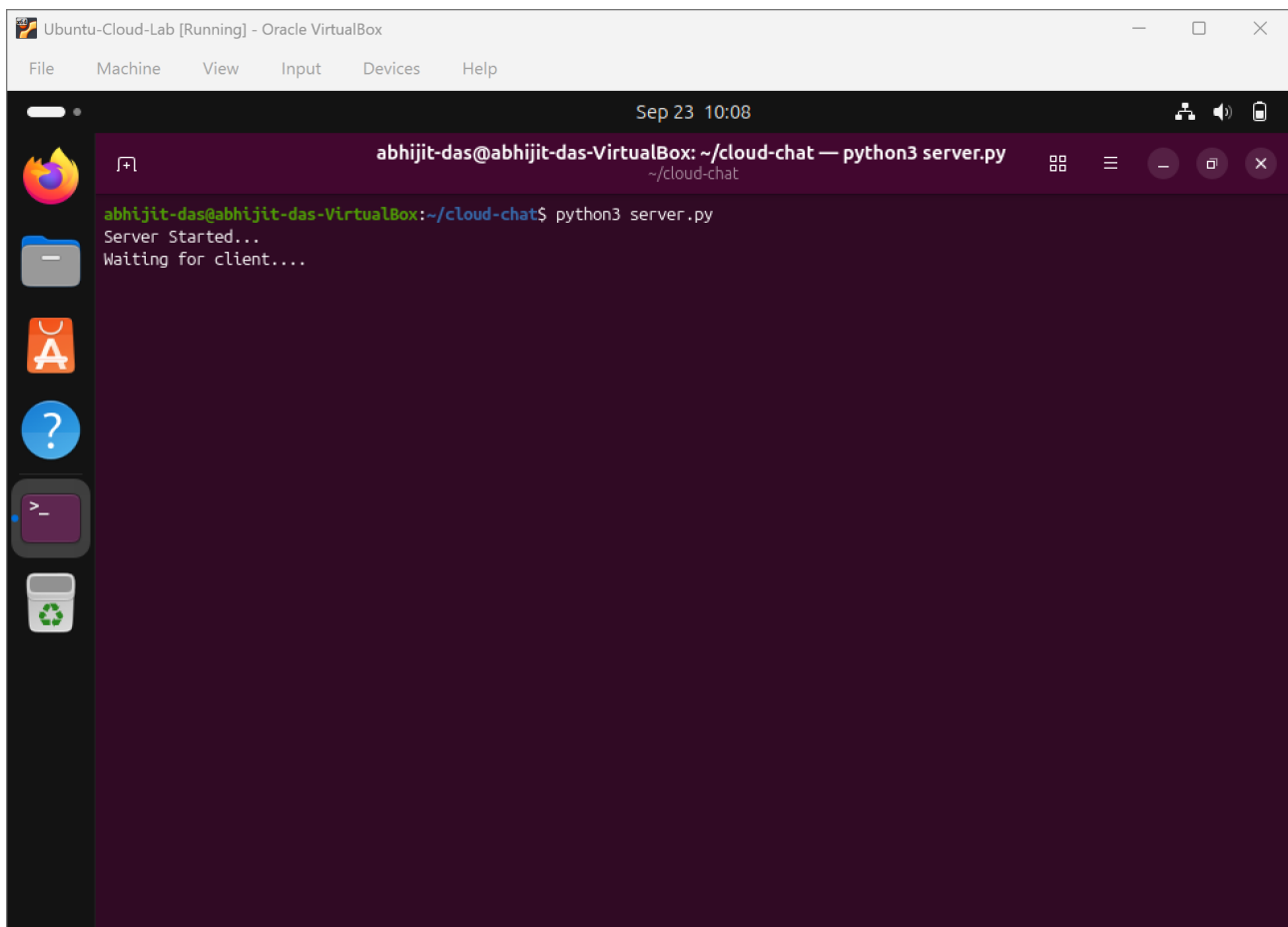


Figure 8: TCP server started and waiting for client connection.

9. Establish the TCP connection

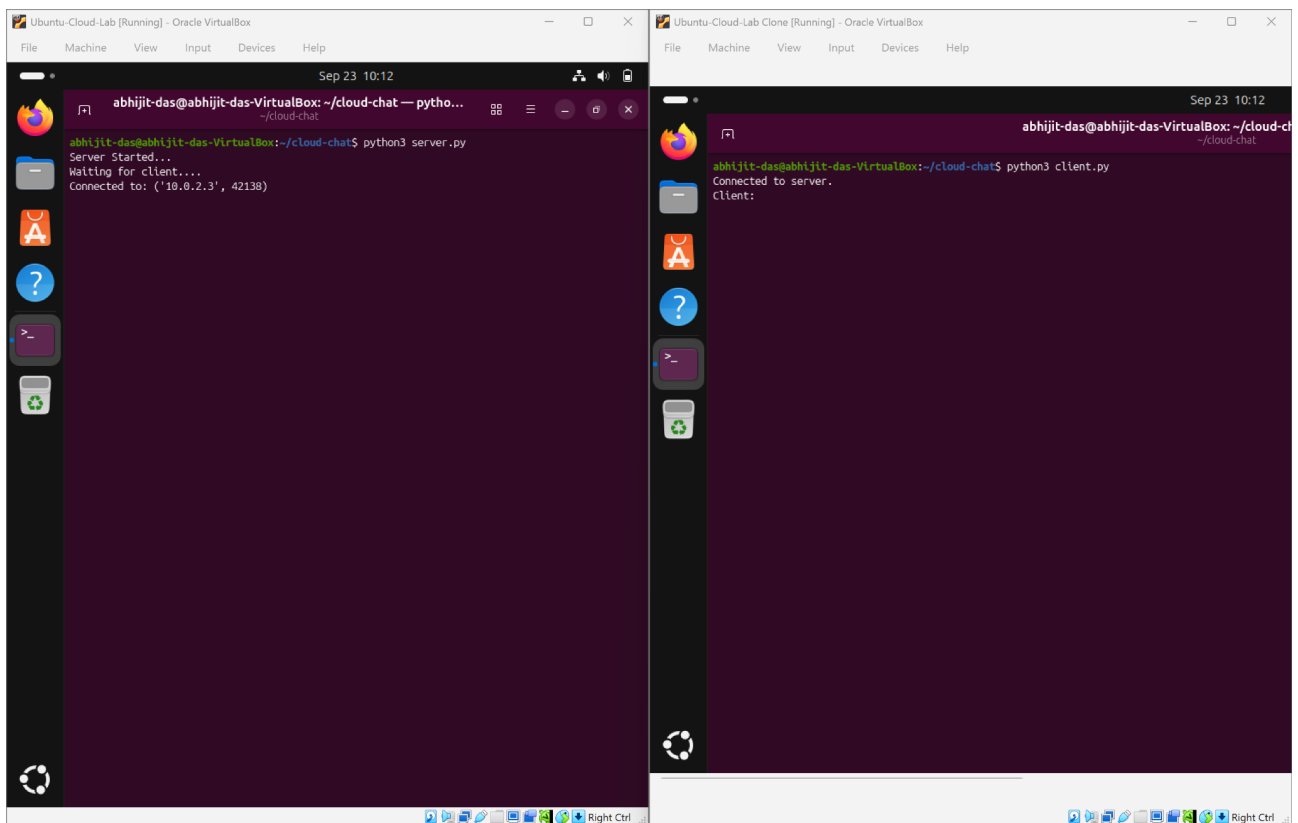
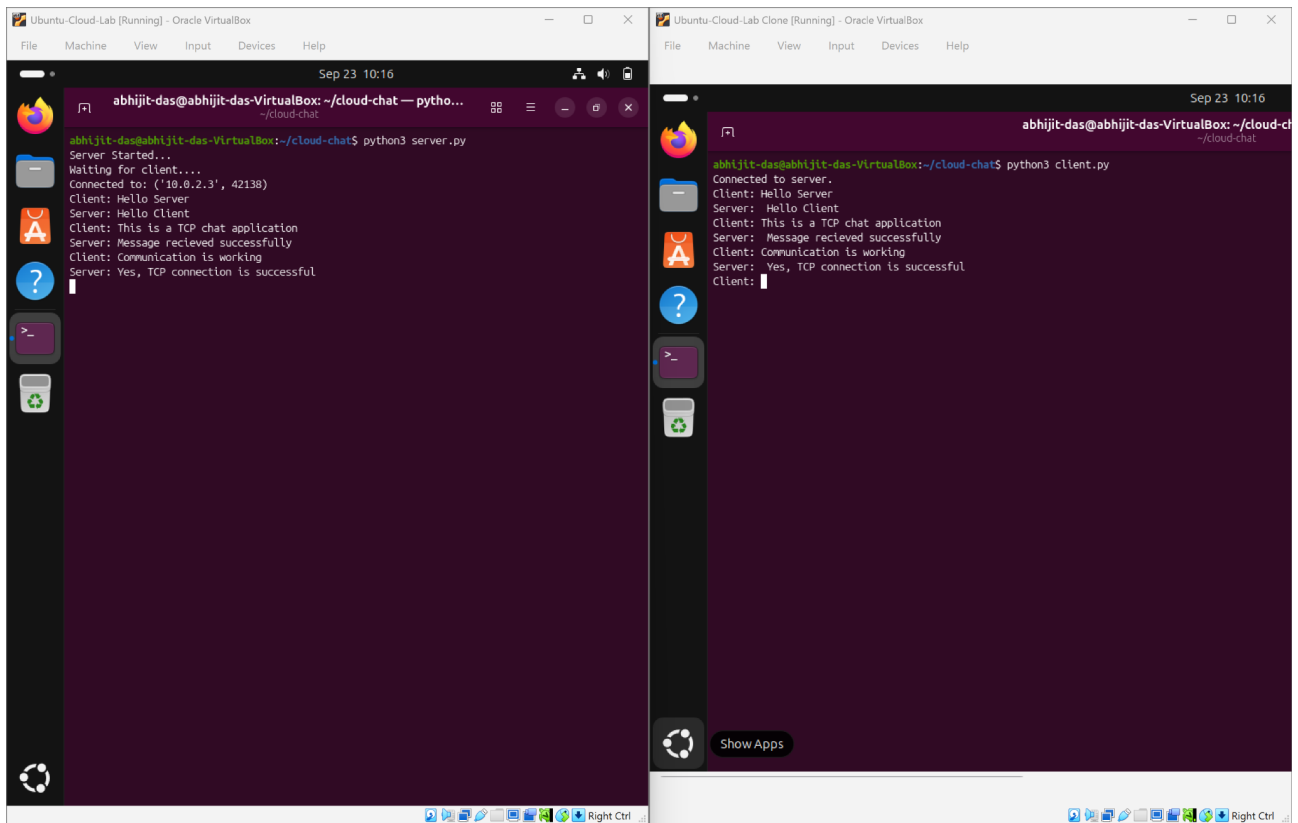


Figure 9: TCP connection established between the Client and Server VMs.

10. Send and receive messages through the chat application



The image displays two side-by-side terminal windows from an Oracle VM VirtualBox environment, showing the execution of a TCP chat application. Both windows are titled 'Ubuntu-Cloud-Lab [Running] - Oracle VirtualBox' and show the user 'abhijit-das' in the directory '~/cloud-chat'.

Left Window (Server): The terminal shows the execution of `python3 server.py`. The output indicates the server has started and is waiting for a client. It then shows a successful connection to '10.0.2.3', 42138. The server receives the message 'Hello Server' from the client, responds with 'Hello Client', and then receives 'This is a TCP chat application'. The server responds with 'Message recieved successfully' and 'Communication is working'. Finally, the server receives 'Yes, TCP connection is successful' from the client.

Right Window (Client): The terminal shows the execution of `python3 client.py`. The output indicates the client has connected to the server. It then sends 'Hello Server' to the server, receives 'Hello client' in response, and sends 'This is a TCP chat application'. The client receives 'Message recieved successfully' and 'Communication is working' from the server. Finally, the client sends 'Yes, TCP connection is successful' to the server, which responds with 'Yes, TCP connection is successful'.

Figure 10: Successful message exchange between Client and Server using TCP.